

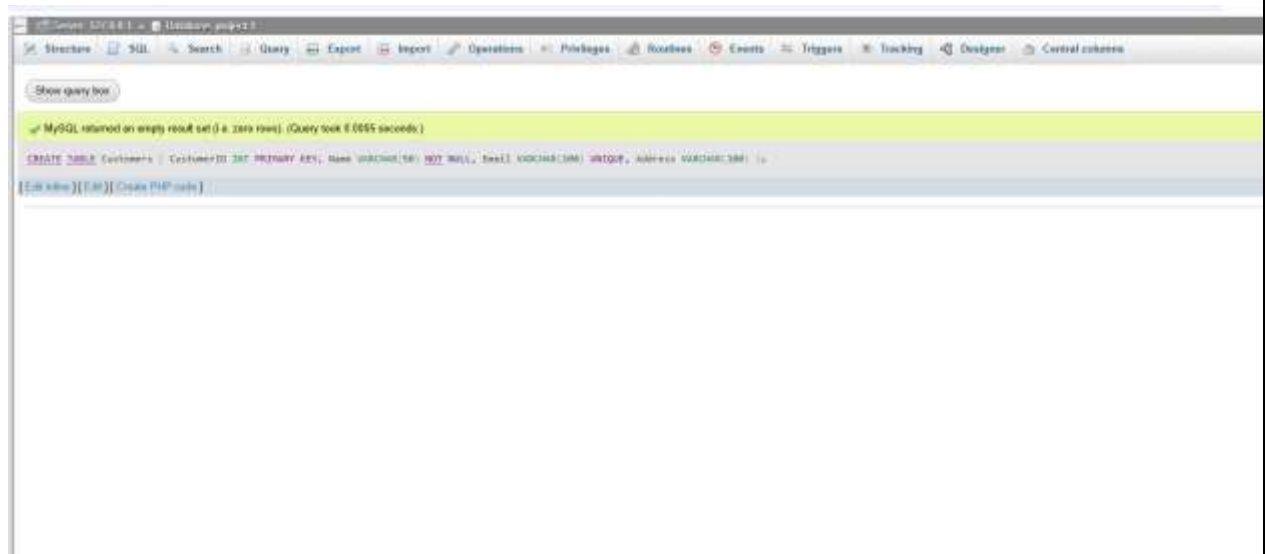
PROJECT-1

ABOUT CUSTOMER:-

1).

CREATE TABLE Customers

```
(
  CustomerID INT PRIMARY KEY,
  Name VARCHAR(50) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  Address VARCHAR(100)
);
```



2)

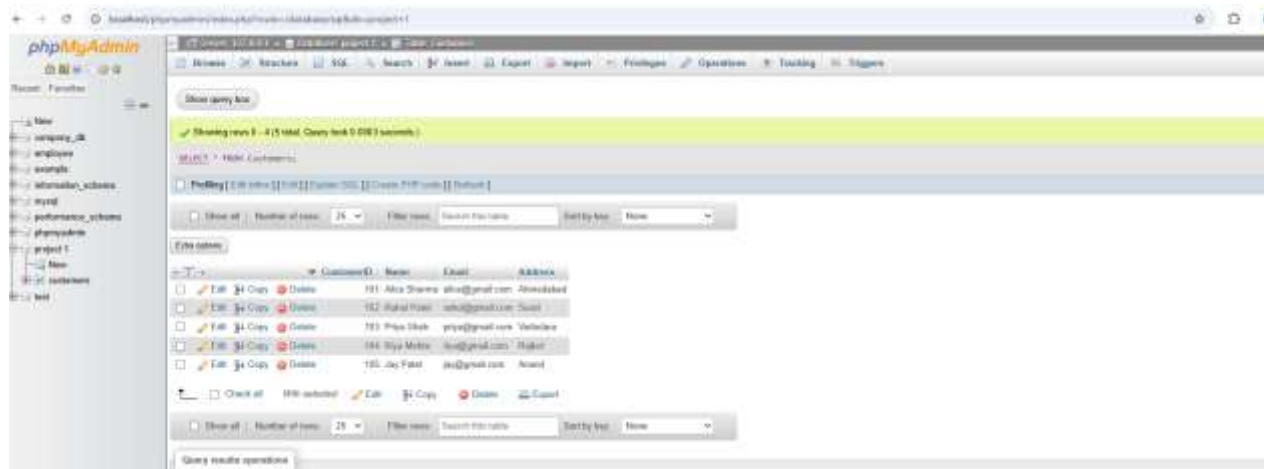
INSERT INTO Customers (CustomerID, Name, Email, Address)
VALUES

```
(101, 'Alice Sharma', 'alice@gmail.com', 'Ahmedabad'),
(102, 'Rahul Patel', 'rahul@gmail.com', 'Surat'),
(103, 'Priya Shah', 'priya@gmail.com', 'Vadodara'),
(104, 'Riya Mehta', 'riya@gmail.com', 'Rajkot'),
(105, 'Jay Patel', 'jay@gmail.com', 'Anand');
```



PROJECT-1

3)
SELECT * FROM Customers;



4)
UPDATE Customers
SET Address = 'Gandhinagar'
WHERE CustomerID = 101;



5)
DELETE FROM Customers
WHERE CustomerID = 105;



6)
SELECT * FROM Customers
WHERE Name = 'Alice Sharma';



PROJECT-1

ORDER :-

1)

CREATE TABLE Orders

(

OrderID INT PRIMARY KEY,

CustomerID INT,

OrderDate DATE,

TotalAmount DECIMAL(10,2),

FOREIGN KEY (CustomerID)

REFERENCES Customers(CustomerID)

ON DELETE CASCADE

);



2)

INSERT INTO Orders (OrderID, CustomerID, OrderDate, TotalAmount)
VALUES

(1001, 101, '2026-09-01', 6000),

(1002, 102, '2026-09-05', 30000),

(1003, 101, '2026-09-10', 55000),

(1004, 103, '2026-09-15', 3500),

(1005, 104, '2026-09-20', 2500);



3) Retrieve all orders made by a specific customer

SELECT *

FROM Orders

WHERE CustomerID = 101;

PROJECT-1

Showing rows 0 - 1 (2 total, Query took 0.0033 seconds.)

SELECT * FROM Orders WHERE CustomerID = 101;

OrderID	CustomerID	OrderDate	TotalAmount
1001	101	2025-20-20	8000.00
1002	101	2025-20-20	9000.00

(4) Calculate overall total order amount

SELECT SUM(TotalAmount) AS Total_Order_Amount
FROM Orders;

Current selection does not contain a unique column. Grid edit, checkbox, Edit, C

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

SELECT SUM(TotalAmount) AS Total_Order_Amount FROM Orders;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

Total_Order_Amount
97000.00

Show all | Number of rows: 25 | Filter rows: Search this table

(5)

Delete an order using OrderID

DELETE FROM Orders

WHERE OrderID = 1005;

1 row deleted. (Query took 0.0019 seconds.)

DELETE FROM Orders WHERE OrderID = 1005;

[Edit inline] [Edit] [Create PHP code]

PROJECT-1

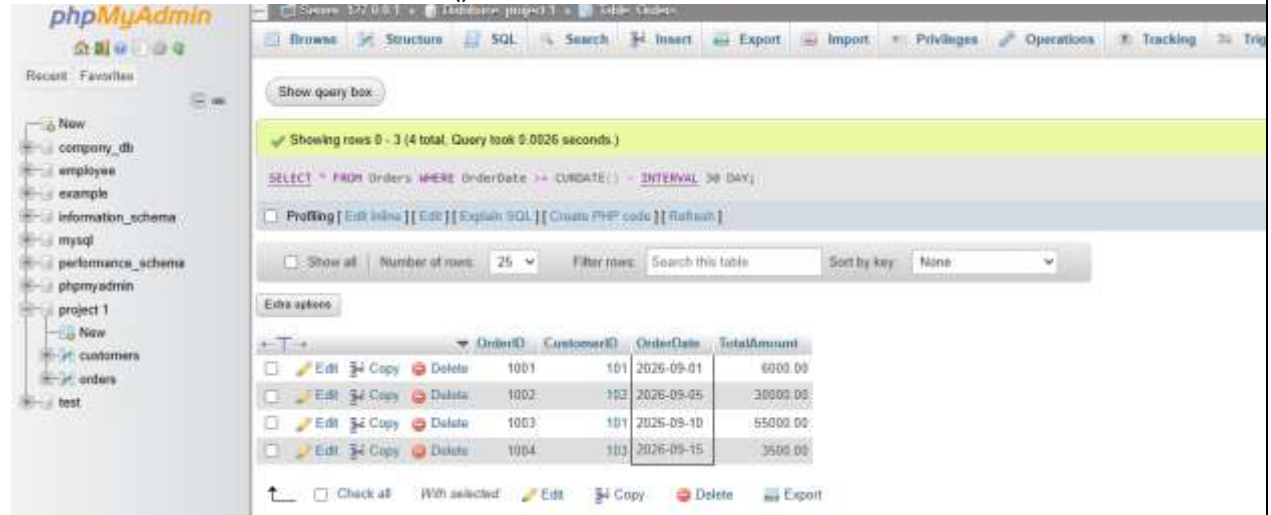
(6)

Retrieve orders placed in the last 30 days

SELECT *

FROM Orders

WHERE OrderDate >= CURDATE() - INTERVAL 30 DAY;



Showing rows 0 - 3 (4 total. Query took 0.0026 seconds.)

```
SELECT * FROM Orders WHERE OrderDate >= CURDATE() - INTERVAL 30 DAY;
```

Number of rows: 25 Filter rows: Search this table Sort by key: None

OrderID	CustomerID	OrderDate	TotalAmount
1001	101	2025-09-01	6000.00
1002	102	2026-09-05	30000.00
1003	101	2025-09-10	55000.00
1004	103	2026-09-15	3500.00

(7)

Find highest, lowest and average order amount

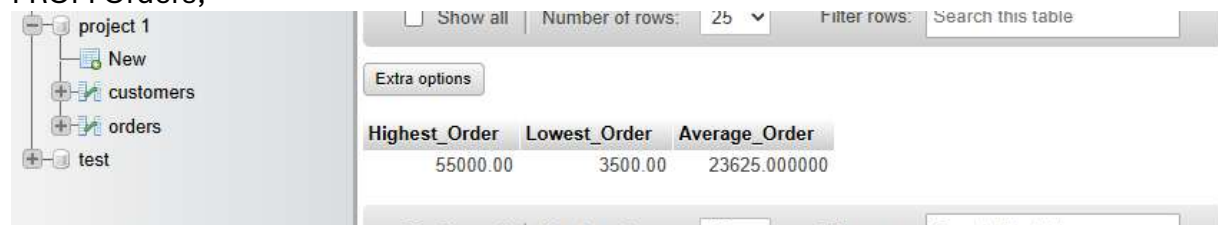
SELECT

MAX(TotalAmount) AS Highest_Order,

MIN(TotalAmount) AS Lowest_Order,

AVG(TotalAmount) AS Average_Order

FROM Orders;



Number of rows: 25 Filter rows: Search this table

Extra options

Highest_Order	Lowest_Order	Average_Order
55000.00	3500.00	23625.000000

PROJECT-1

Products Table:-

(1) CREATE TABLE Products

(
(

ProductID INT PRIMARY KEY,
 ProductName VARCHAR(100) NOT NULL,
 Price DECIMAL(10,2),
 Stock INT
);

(2) INSERT INTO Products (ProductID, ProductName, Price, Stock)
VALUES

(201, 'Laptop', 55000, 10),
 (202, 'Smartphone', 30000, 15),
 (203, 'Headphones', 3500, 20),
 (204, 'Keyboard', 2500, 25),
 (205, 'Mouse', 1200, 0);



(3) Retrieve products sorted by price in descending order

SELECT *
 FROM Products
 ORDER BY Price DESC;



PROJECT-1

(4) Display price of a specific product

```
SELECT ProductName, Price
```

```
FROM Products
```

```
WHERE ProductName = 'Laptop';
```



The screenshot shows a database management tool interface. On the left is a tree view with 'project 1' expanded, containing 'New', 'customers', 'orders', 'products', and 'test'. The main area displays a query result table with columns 'ProductName' and 'Price'. The row for 'Laptop' shows a price of 55000.00. Below the table are buttons for 'Edit', 'Copy', and 'Delete'. At the bottom, there are checkboxes for 'Check all' and 'With selected:', followed by 'Edit', 'Copy', 'Delete', and 'Export' buttons.

ProductName	Price
Laptop	55000.00

(5) Delete a product if it is out of stock

```
DELETE FROM Products
```

```
WHERE Stock = 0;
```



The screenshot shows a database management tool interface. A yellow banner at the top indicates '1 row deleted: (Query took 0.0002 seconds)'. Below this, the SQL query 'DELETE FROM Products WHERE Stock = 0;' is displayed. At the bottom, there are buttons for 'Edit', 'Copy', and 'Delete'.

(6) Retrieve products whose price is between ₹2500 and ₹5000

```
SELECT *
```

```
FROM Products
```

```
WHERE Price BETWEEN 2500 AND 5000;
```



The screenshot shows a database management tool interface. The main area displays a query result table with columns 'ProductID', 'ProductName', 'Price', and 'Stock'. Two rows are shown: '203 Headphones' with a price of 3500.00 and stock of 20, and '204 Keyboard' with a price of 2500.00 and stock of 25. Below the table are buttons for 'Edit', 'Copy', and 'Delete'. At the bottom, there are checkboxes for 'Check all' and 'With selected:', followed by 'Edit', 'Copy', 'Delete', and 'Export' buttons.

ProductID	ProductName	Price	Stock
203	Headphones	3500.00	20
204	Keyboard	2500.00	25

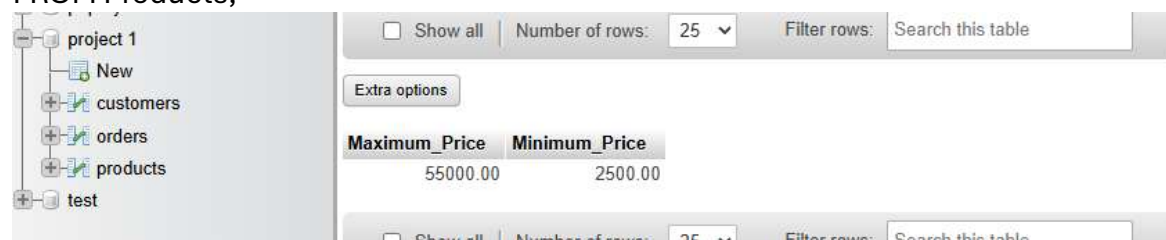
(7) Find the most expensive and cheapest product using MAX() and MIN()

```
SELECT
```

```
MAX(Price) AS Maximum_Price,
```

```
MIN(Price) AS Minimum_Price
```

```
FROM Products;
```



The screenshot shows a database management tool interface. The main area displays a query result table with columns 'Maximum_Price' and 'Minimum_Price'. The values are 55000.00 and 2500.00 respectively. Above the table are buttons for 'Show all', 'Number of rows: 25', and 'Filter rows: Search this table'. Below the table are buttons for 'Edit', 'Copy', and 'Delete'. At the bottom, there are checkboxes for 'Check all' and 'With selected:', followed by 'Edit', 'Copy', 'Delete', and 'Export' buttons.

Maximum_Price	Minimum_Price
55000.00	2500.00

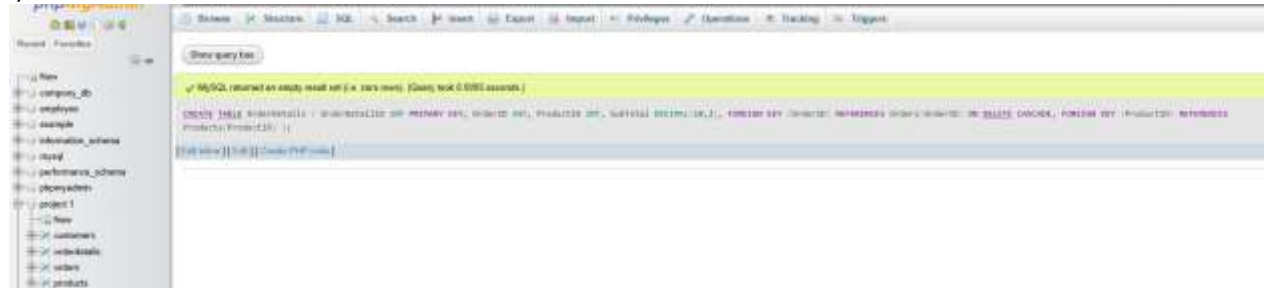
PROJECT-1

OrderDetails Table:-

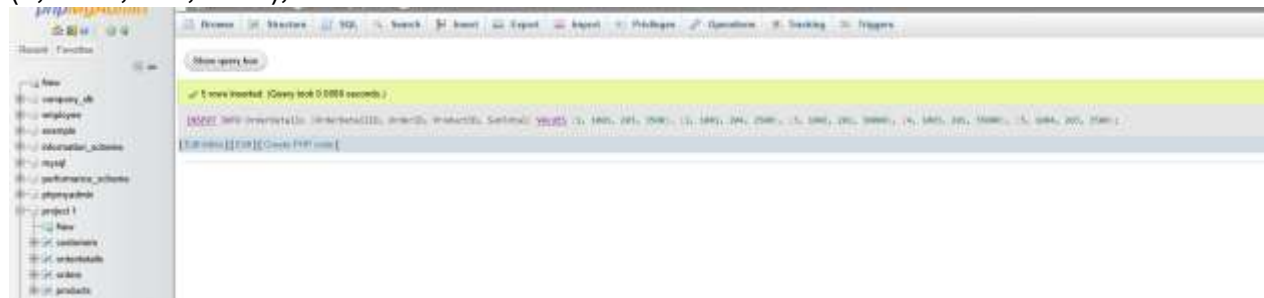
```
(1)CREATE TABLE OrderDetails
(
    OrderDetailID INT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    SubTotal DECIMAL(10,2),

    FOREIGN KEY (OrderID)
    REFERENCES Orders(OrderID)
    ON DELETE CASCADE,

    FOREIGN KEY (ProductID)
    REFERENCES Products(ProductID)
);
```



```
(2) INSERT INTO OrderDetails
(OrderDetailID, OrderID, ProductID, SubTotal)
VALUES
(1, 1001, 203, 3500),
(2, 1001, 204, 2500),
(3, 1002, 202, 30000),
(4, 1003, 201, 55000),
(5, 1004, 203, 3500);
```



```
(3) Retrieve all order details for a specific order
SELECT *
FROM OrderDetails
WHERE OrderID = 1001;
```



PROJECT-1

(4) Calculate total revenue generated from all orders

```
SELECT SUM(SubTotal) AS Total_Revenue
FROM OrderDetails;
```

Total_Revenue
94500.00

(5) Retrieve the top 3 most ordered products

```
SELECT
    ProductID,
    COUNT(*) AS Order_Count
FROM OrderDetails
GROUP BY ProductID
ORDER BY Order_Count DESC
LIMIT 3;
```

ProductID	Order_Count
203	2
201	1
202	1

(6) Count how many times a specific product has been sold

```
SELECT
    ProductID,
    COUNT(*) AS Times_Sold
FROM OrderDetails
WHERE ProductID = 203
GROUP BY ProductID;
```

ProductID	Times_Sold
203	2

PROJECT-1

PROJECT-1